Лабораторна робота №2

Паралелізм даних та паралелізм задач

Виконала студентка групи МІТ-41

Пась Олександра

Хід виконання

1. Реалізуйте програму на C#, яка створює дві задачі (Task) для паралельного
виконання:

- перша задача виводить числа від 1 до 10 з паузою 200 мс;
- друга задача виводить літери від A до J з паузою 200 мс.

```
static void Main()
{
    Console.WriteLine("Головний потік починає роботу.");

    // Створення першої задачі: виводить числа від 1 до 10 з паузою 200 мс
    Task task1 = new Task(() =>
    {
        for (int i = 1; i <= 10; i++)
        {
            Console.WriteLine($"Задача 1: число {i}");
            Thread.Sleep(200);
        }
    });

    // Створення другої задачі: виводить літери від A до J з паузою 200 мс
    Task task2 = new Task(() =>
    {
        for (char c = 'A'; c <= 'J'; c++)
        {
            Console.WriteLine($"Задача 2: літера {c}");
            Thread.Sleep(200);
        }
    });

    // Запуск обох задач
    task1.Start();
    task2.Start();

    // Очікування завершення обох задач
    Task.WaitAll(task1, task2);

    Console.WriteLine("Головний потік завершив роботу.");
}
```

рис. 2.1 - код завдання 1

```
Головний потік починає роботу.
Задача 1: число 1
Задача 2: літера A
Задача 1: число 2
Задача 2: літера B
Задача 1: число 3
Задача 2: літера C
Задача 1: число 4
Задача 2: літера D
Задача 1: число 5
Задача 2: літера E
Задача 1: число 6
Задача 2: літера F
Задача 1: число 7
Задача 2: літера G
Задача 1: число 8
Задача 2: літера H
Задача 1: число 9
Задача 2: літера I
Задача 1: число 10
Задача 2: літера J
Головний потік завершив роботу.
```

рис. 2.2 - результат виконання

Поясніть різницю між класами Thread та Task.

Thread — це низькорівневий клас для створення окремого потоку ОС. Кожен Thread займає ресурси системи (стек, контекст), і їх створення є відносно дорогим. Керування потоками здійснюється вручну.

Task — це високорівнева абстракція з бібліотеки TPL, яка представляє асинхронну операцію. Задачі виконуються пулом потоків (ThreadPool), що дозволяє ефективно використовувати ресурси. Вони підтримують зручні механізми: продовження, очікування, повернення результату, обробка помилок тощо.

2. Створіть програму, яка запускає три задачі, кожна з яких виконує підрахунок до 5. Виведіть у консоль ідентифікатор (Id) кожної задачі та поточний ідентифікатор (CurrentId) під час виконання.

```
static void Main()
{
    Console.WriteLine("Головний потік починає роботу.");
    Console.WriteLine($"Головний потік ID: {Thread.CurrentThread.ManagedThreadId}");
    Console.WriteLine();

    // Створюємо три задачі
    Task task1 = new Task(CountToFive);
    Task task2 = new Task(CountToFive);
    Task task3 = new Task(CountToFive);

    // Виводимо Id задач до запуску
    Console.WriteLine($"Id задачі 1 до запуску: {task1.Id}");
    Console.WriteLine($"Id задачі 2 до запуску: {task2.Id}");
    Console.WriteLine($"Id задачі 3 до запуску: {task3.Id}");
    Console.WriteLine();

    // Запускаємо задачі
    task1.Start();
    task2.Start();
    task3.Start();

    // Очікуємо завершення всіх задач
    Task.WaitAll(task1, task2, task3);

    Console.WriteLine();
    Console.WriteLine("Всі задачі завершено.");
    Console.WriteLine("Головний потік завершив роботу.");
}

3 references
static void CountToFive()
{
    // Виводимо Id задачі та поточний потік
    Console.WriteLine($"Задача {Task.CurrentId} стартувала на потоці {Thread.CurrentThread.ManagedThreadId}");

    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine($"Задача {Task.CurrentId}: лічба = {i}");
        Thread.Sleep(300); // Невелика затримка, щоб побачити паралелізм
    }

    Console.WriteLine($"Задача {Task.CurrentId} завершилась");
}
```

рис. 2.3 - код завдання

```
Головний потік починає роботу.
Головний потік ID: 1

Id задачі 1 до запуску: 8
Id задачі 2 до запуску: 9
Id задачі 3 до запуску: 10

Задача 8 стартувала на потоці 9
Задача 8: лічба = 1
Задача 9 стартувала на потоці 8
Задача 9: лічба = 1
Задача 10 стартувала на потоці 10
Задача 10: лічба = 1
Задача 9: лічба = 2
Задача 10: лічба = 2
Задача 8: лічба = 2
Задача 8: лічба = 3
Задача 10: лічба = 3
Задача 9: лічба = 3
Задача 8: лічба = 4
Задача 10: лічба = 4
Задача 9: лічба = 4
Задача 8: лічба = 5
Задача 10: лічба = 5
Задача 9: лічба = 5
Задача 8 завершилась
Задача 10 завершилась
Задача 9 завершилась

Всі задачі завершено.
```

рис. 2.4 - результат виконання

Чи є ідентифікатори задач впорядкованими? Як їх можна використати для відстеження виконання?

Ідентифікатори задач (Task.Id) не є впорядкованими — вони просто присвоюються унікально для кожної задачі і не залежать від порядку їх створення чи виконання.

Їх можна використовувати для відстеження виконання задач: наприклад, під час роботи програми виводити Task.Id у консоль, щоб зрозуміти, яка саме задача зараз виконується, у якій послідовності вони стартують і завершуються. Це допомагає аналізувати паралельне виконання і відлагоджувати програму.

3. Реалізуйте програму, яка:

- створює задачу для обчислення суми чисел від 1 до N (N вводиться користувачем).
- використовує метод ContinueWith() для запуску другої задачі, яка виводить
результат у консоль.
- використовує лямбда-вираз для реалізації продовження.

```
class Program
{
    0 references
    static void Main()
    {
        Console.WriteLine("Головний потік починає роботу.");

        // Введення числа N користувачем
        Console.Write("Введіть число N: ");
        int N = int.Parse(Console.ReadLine());

        // 1. Створюємо задачу для обчислення суми чисел від 1 до N
        Task<long> sumTask = new Task<long>(() =>
        {
            long sum = 0;
            for (int i = 1; i <= N; i++)
            {
                sum += i;
            }
            return sum;
        });

        // 2. Використовуємо ContinueWith() для запуску другої задачі
        // 3. Лямбда-вираз для реалізації продовження
        Task continueTask = sumTask.ContinueWith((previousTask) =>
        {
            long result = previousTask.Result;
            Console.WriteLine($"Сума чисел від 1 до {N} = {result}");
        });

        // Запускаємо першу задачу
        sumTask.Start();

        // Очікуємо завершення продовження
        continueTask.Wait();

        Console.WriteLine("Головний потік завершив роботу.");
    }
}
```

рис. 2.5 - код завдання

```
Головний потік починає роботу.
Введіть число N: 4
Сума чисел від 1 до 4 = 10
Головний потік завершив роботу.
```

рис. 2.6 - результат виконання

Які переваги має використання продовжень задач у порівнянні з класичним підходом синхронізації?

Продовження задач простіші й зручніші за класичну синхронізацію, бо дозволяють запускати наступну дію автоматично після завершення попередньої задачі. Це зменшує потребу в блокуваннях (lock, Wait) і складному керуванні потоками. Код стає більш читабельним, логіка виконання — чіткішою, а також краще підтримується асинхронність без зайвого простою потоків.

4. Розробіть програму, яка виконує три методи паралельно за допомогою Parallel.Invoke():

- метод 1: обчислює факторіал числа.
- метод 2: обчислює суму чисел від 1 до N.
- метод 3: виводить повідомлення у консоль з паузою 300 мс.

```
class Program
{
    0 references
    static void Main()
    {
        Console.WriteLine("Головний потік починає роботу.");
        Console.WriteLine($"Id головного потоку: {Thread.CurrentThread.ManagedThreadId}\n");

        // Введення даних користувачем
        Console.Write("Введіть число для обчислення факторіала: ");
        int factNumber = int.Parse(Console.ReadLine());

        Console.Write("Введіть число N для обчислення суми від 1 до N: ");
        int sumNumber = int.Parse(Console.ReadLine());

        Console.WriteLine("\nЗапуск паралельного виконання методів...\n");

        // Паралельне виконання трьох методів
        Parallel.Invoke(
            () => Method1_Factorial(factNumber),
            () => Method2_Sum(sumNumber),
            () => Method3_Message()
        );

        Console.WriteLine("\nВсі методи завершили виконання.");
        Console.WriteLine("Головний потік завершив роботу.");
    }

    // Метод 1: обчислення факторіала числа
    1 reference
    static void Method1_Factorial(int n)
    {
        Console.WriteLine($"[{DateTime.Now:HH:mm:ss.fff}] Метод 1 (факторіал) стартував на потоці {Thread.CurrentThread.ManagedThreadId}");

        long factorial = 1;
        for (int i = 1; i <= n; i++)
        {
            factorial *= i;
            Thread.Sleep(100); // Невелика затримка для наочності
        }
```

рис. 2.7 - частина коду

```
Головний потік починає роботу.
Id головного потоку: 1

Введіть число для обчислення факторіала: 5
Введіть число N для обчислення суми від 1 до N: 24

Запуск паралельного виконання методів...

[20:18:25.139] Метод 3 (повідомлення) стартував на потоці 8
[20:18:25.200] Метод 3: Перше повідомлення
[20:18:25.140] Метод 2 (сума) стартував на потоці 9
[20:18:25.139] Метод 1 (факторіал) стартував на потоці 1
[20:18:25.515] Метод 3: Друге повідомлення
[20:18:25.764] Метод 1: факторіал числа 5 = 120
[20:18:25.828] Метод 3: Третє повідомлення
[20:18:26.134] Метод 3: Четверте повідомлення
[20:18:26.440] Метод 3: П'яте повідомлення
[20:18:26.741] Метод 3 завершив виведення повідомлень
[20:18:27.819] Метод 2: сума від 1 до 24 = 300

Всі методи завершили виконання.
Головний потік завершив роботу.
```

рис. 2.8 - результат виконання

Які переваги має використання Parallel.Invoke() у порівнянні зі створенням окремих задач?
Parallel.Invoke() простіший у використанні, ніж створення окремих задач через Task. Він дозволяє запустити кілька методів паралельно однією командою, без ручного створення і керування задачами (Task.Run, Wait, тощо). Також Parallel.Invoke() автоматично використовує пул потоків і сам оптимізує розподіл навантаження. Тому код стає коротшим і менш схильним до помилок, хоча він менш гнучкий, ніж робота з Task.

Висновок: У ході виконання лабораторної роботи я практично ознайомилася з основними можливостями бібліотеки TPL для паралельного програмування в C#. Було створено програми з використанням задач (Task), методів Start(), WaitAll() та ContinueWith(), а також досліджено паралельне виконання методів за допомогою Parallel.Invoke()..